Question - 1
Meeting Scheduler

You are given *m* distinct rooms (numbered *1, 2....m)* and a list of *n* existing bookings for different rooms. Given a requested meeting window, return list of all rooms in a sorted order that have no meetings scheduled at any moment in the requested meeting window.

Note : A meeting ending at time *t* does not overlap with another meeting starting at *t*.

Inputs:

- *requestedStartTime, requestedEndTime* : start time and end time of the requested meeting window
- *m* : total meeting rooms available
- *existingBookings* : each entry denotes an existing booking
  - *existingBookings[i][0]* : denotes room ID of ith scheduled meeting
  - *existingBookings[i][1]* : denotes start time of ith scheduled meeting
  - *existingBookings[i][2]* : denotes end time of ith scheduled meeting

Example

Input : *requestedStartTime* = 5, *requestedEndTime* = 10, *m = 4, existingBookings* = [[1, 3, 5], [1, 1, 2], [2, 2, 6]]

Output : [1, 3, 4]

- The meeting roomID 1 has meetings at [1, 2] and [3, 5]. This does not overlap with the requested window of [5, 10].
- RoomID 2 has a meeting at [2, 6] and this overlaps with requested window [5, 10] at time 5.
- RoomIDs 3 and 4 have no meetings scheduled.
- Hence, roomIDs {1, 3, 4} are available.

Constraints

- $1 \le m, n \le 2 * 10^5$
- $1 \le requestedStartTime, requestedEndTime \le 10^9$
- $1 \le existingBookings[0] \le 2 * 10^5$
- $1 \le existingBookings[1] < existingBookings[2] \le 10^9$
- Existing meetings in the same room are guaranteed not to overlap or conflict.

Question - 2
Most Occurring Element

Given an array, find the most frequently occurring element in the array. If multiple elements have the same maximum occurring frequency, find the largest of them.

Example 1

input: *arr* = [1, 1, 2, 3]

Output: 1

Explanation:

Elements and their frequencies

Element	Frequency
1	2
2	1
3	1

Element 1 has a maximum frequency of 2.

Example 2
input: *arr* = [1, 1, 2, 2, 3]
Output: 2
Explanation:

Elements and their frequencies

Elements	Frequency
1	2
2	2
3	1

Element values 1 and 2 share the maximum frequency of 2.
Return the greater value in this case of ties.

Constraints

- $1 \le n \le 2 * 10^5$
- $0 \le arr[i] \le 10^9$

Question - 3
Count of Palindromic Substrings

Given a string, find the count of all substrings of the string that are palindromes.

Note: A palindrome is a string that reads the same from the first character to the last and from the last character to the first.

Example 1
Input: *s* = "abcbd"
Output: 6
Explanation: The palindromic substrings of the string s are "a," "b," "c," "b," "d," and "bcb."

Example 2
Input: *s* = "abdaa"
Output: 6
Explanation: The palindromic substrings of the string s are "a," "b," "d," "a," "a," and "aa."

Constraints

- $1 \le |s| \le 2000$
- It is guaranteed that the string *s* consists of lowercase English letters.

Question - 4
Increasing Pairs

Given an array of *n* integers, return the number of strictly increasing subsequences of length 2.

Note: A subsequence is a sequence that can be derived from an array by deleting zero or more elements without changing the order of the remaining elements.

Example 1
Input: *n* = 3, *arr* = [1, 2, 1]
Output: 1

All subsequences of length two(0-based indexing)

Index of First Element	Index of Second Element	Is Strictly Increasing?
0	1	Yes
0	2	No
1	2	No

There is only one increasing subsequence of length 2: {1, 2}.

Example 2
Input: *n* = 4, *arr* = [3, 1, 4, 2]
Output: 3
The valid increasing subsequences of length 2 are: {3, 4}, {1, 4}, and {1, 2}.

Constraints
- $1 \le n \le 5000$
- $0 \le arr[i] \le 10^9$

Question - 5
Strictly Increasing Triplets

Given an array of integers, return the number of strictly increasing subsequences of length 3 (modulo (10^9+7)).

Note: A subsequence is a sequence that can be derived from an array by deleting zero or more elements without changing the order of the remaining elements.

Example 1
Input: *n* = 5, *arr* = [1, 2, 3, 4, 1]
Output: 4

Some subsequences of length 3 (0-based indexing)

Index of 1st Element	Index of 2nd Element	Index of 3rd Element	Subsequence	Is Strictly Increasing?
0	1	2	[1, 2, 3]	Yes
0	2	4	[1, 3, 1]	No
0	1	3	[1, 2, 4]	Yes

0	2	3	[1, 3, 4]	Yes
1	2	4	[2, 3, 1]	No
1	2	3	[2, 3, 4]	Yes

All strictly increasing subsequences of length three are: [1, 2, 3], [1, 2, 4], [1, 3, 4], and [2, 3, 4].
The answer is 4 modulo (10^9+7), which is 4.

Example 2
Input: *n* = 4, *arr* = [3, 1, 4, 5]
Output: 2
There are two strictly increasing subsequences of length three: [3, 4, 5] and [1, 4, 5].

Constraints

- $1 \le n \le 5000$
- $0 \le arr[i] \le 10^9$

Question - 6
Missing Prefix Positive Sum

Given an array of integers, for every prefix of the array, find the smallest positive integer that does not appear in that prefix.

Compute the sum of these values across every possible prefix of *arr*. Return this sum modulo 10^9+7.

Note: Prefix is a contiguous segment starting from the first element of the array up to a specified index.

Example 1
Input: *arr* = [1, 2, 0]
Output: 8
Explanation:
We look at every prefix:

1. Prefix: [1]
    - Positive numbers present: 1
    - The smallest missing positive is 2.
2. Prefix: [1, 2]
    - Positive numbers present: 1, 2
    - The smallest missing positive is 3.
3. Prefix: [1, 2, 0]
    - Positive numbers present: 1, 2
    - The smallest missing positive is 3.

Adding the results of all prefixes:
2 + 3 + 3 = 8
Thus, the function should return 8 modulo (10^9 + 7), which is 8.

Example 2
Input: *arr* = [3, 4, 1, -1]
Output: 6
Explanation:
We look at every prefix:

1. Prefix: [3]
    - Positive numbers present: 3

- The smallest missing positive is 1.

2. Prefix: [3, 4]
   - Positive numbers present: 3, 4
   - The smallest missing positive is 1.
3. Prefix: [3, 4, 1]
   - Positive numbers present: 1, 3, 4
   - The smallest missing positive is 2.
4. Prefix: [3, 4, 1, -1]
   - Positive numbers present: 1, 3, 4
   - The smallest missing positive is 2.

Adding the results of all prefixes:

1 + 1 + 2 + 2 = 6

Thus, the function should return 6 modulo (10^9 + 7), which is 6.

Constraints

- $0 \le n \le 2 * 10^5$
- $-10^9 \le arr[i] \le 10^9$

Question - 7

Equal Frequency Removal

Given an array of integers, in one operation, you may choose one specific value and delete all its occurrences. After performing any number of such deletions, the frequencies of all remaining distinct elements must be equal.

Find the minimum number of operations required to achieve this goal.

Example 1

Input: *arr* = [1, 2, 2, 3, 3, 3]

Output: 2

Explanation:

The set of distinct elements in the array = [1, 2, 3]

Considering different deletion possibilities

Elements to be removed	Number of Operations	Array after operations	Goal achieved?
-	0	[1, 2, 2, 3, 3, 3]	No
1	1	[2, 2, 3, 3, 3]	No
2	1	[1, 3, 3, 3]	No
3	1	[1, 2, 2]	No
1, 2	2	[3, 3, 3]	Yes
1, 3	2	[2, 2]	Yes
2, 3	2	[1]	Yes
1, 2, 3	3	[]	Yes

One optimal solution to the problem is to remove all occurrences of 2 and 3.

This results in the singleton array [1].

Example 2

Input: *arr* = [1, 2, 2, 2, 3, 3, 3, 4]
Output: 2
Explanation:
One optimal solution to the problem is to remove all occurrences of 2 and 3.
This results in the array [1, 4].

Constraints

- $0 \le n \le 2 * 10^5$
- $0 \le arr[i] \le 10^6$

Question - 8
Minimum Alternating Flips

You are given a binary string; transform it into an alternating binary string using the following operation: select any bit and flip it (change 0 → 1 or 1 → 0).

Find the minimum number of operations required to make the string alternating.

Example 1
Input: *s* = "11101"
Output: 1
Explanation:

Possible ways of performing operations

Positions of bits to be flipped (following 0-based indexing)	String after operations	Is string alternating?
0, 2, 3, 4	"01010"	Yes
1	"10101"	Yes
0, 3	"01111"	No

An optimal way to make the string alternating is to flip the bit at position 1 (0-based indexing). Thus, only one operation is needed.

Example 2
Input: s = "1111101"
Output: 2
Explanation:
We can perform the operation on the 1st and 3rd characters to make the string alternating. This results in the string "1010101".

Constraints

- $0 \le$ length of $s \le 2 * 10^5$
- string s consists of characters '0' and '1'

Question - 9
Longest Alternating Flip

Given a binary string, you can perform at most one flip (change 0 → 1 or 1 → 0 at any position). Find the maximum length of a substring consisting of alternating characters after at most one flip.

Example 1
Input: *s* = "11101"

Output: 5
Explanation:

Possible ways to perform operations

Positions of bit to be flipped (following 0-based indexing)	String after operations	Splitting string to alternating substrings
-	"11101"	"1", "1", "101"
1	"10101"	"10101"
2	"11001"	"1", "10", "01"

Flipping the bit at index 1 (0-based) transforms the string into "10101", which is a fully alternating substring of length 5, the longest possible in this case.

Example 2
Input: *s* = "1111101"
Output: 5
Explanation:
Without performing any operation, the length of the longest alternating substring is 3, which is "101".
If the operation is performed on the 3rd character. This results in the string "1110101", which has an alternating substring of size 5, "10101".

Constraints

- 0 ≤ length of *s* ≤ 5000
- string *s* consists of characters '0' and '1'

Question - 10
Common Prefix

You are given a list of invoice IDs represented as numeric strings.
Find the longest prefix that appears in at least the given minimum number of invoices. If there are multiple answers with the same length, return the lexicographically smallest one.

Note: A string *c* is lexicographically smaller than a string *d* if and only if one of the following holds:

- *c* is a prefix of *d*, but *c* does not equal *d*; or
- In the first position where *c* and *d* differ, the digit in *c* is smaller than the digit in *d*.

Example 1
Input: *invoices* = ["10001", "10009", "10111", "10100"], *minNumber* = 2
Output: "1000"
Explanation:

Counts of some prefixes

Prefix	Count
"101"	2
"1"	4
"10"	4
"100"	2
"1000"	2

The longest prefix that appears in at least 2 invoice IDs is "1000".

Example 2

Input: *invoices* = ["123", "121", "122"], *minNumber* = 3

Output: "12"

Explanation:

Counts of some prefixes

Prefix	Count
""	3
"1"	3
"12"	3
"123"	1
"121"	1

The longest prefix that appears in at least 3 invoice IDs is "12".

Constraints

- 1 ≤ lengths of *invoices, invoices[i]* ≤ $4 * 10^2$
- 1 ≤ *minNumber* ≤ length of *invoices*
- All *invoices[i]* contain only characters '0'–'9' and are of equal length.

Question - 11

API Retry Analyzer

Given arrays of timestamps and request IDs, find the total number of retries across all request IDs. A retry is defined as two consecutive logs for the same request ID recorded with a time gap of less than or equal to *gap* seconds. The timestamps array is sorted in non-decreasing order.

The function *getRetryCount* will take three inputs:

int gap: the maximum allowed time difference (in seconds) to consider a retry

string requestIds[n]: request ID for each log

int timestamps[n]: timestamp for each log

The function should return an integer denoting the total number of retries made by all request IDs.

Example

gap = 10

requestIds = ["r1", "r1", "r1", "r2", "r2"]

timestamps = [100, 105, 200, 300, 302]

The table below shows the total number of retries for each request ID:

Request Retry Table

Request ID	Timestamps	Retry Pairs	Retry Count
r1	[100, 105, 200]	(100, 105)	1

r2	[300, 302]	(300, 302)	1

The total number of retries = 1 + 1 = 2.
Hence, the answer is 2.

Constraints

- $1 \le gap \le 10^9$
- $1 \le n \le 2 * 10^5$
- $0 \le timestamps[i] \le 10^9$
- $1 \le$ *length of requestIds[i]* ≤ 10
- It is guaranteed that *requestIds[i]* is an alphanumeric string.
- It is guaranteed that the *timestamps* array is sorted in non-decreasing order.

▼ Input Format for Custom Testing

The first line contains an integer *gap*.
The second line contains an integer *n*, the number of events.
The next *n* lines each contain a string *requestIds[i]*.
The next line contains an integer *n*, the size of *timestamps*.
The next *n* lines each contain an integer *timestamps[i]*.

▼ Sample Case 0

Sample Input 0

```
STDIN          FUNCTION
-----          --------
15      →      gap = 15
6       →      requestIds[] size n = 6
r2      →      requestIds = ["r2", "r1", "r2", "r1", "r2", "r3"]
r1
r2
r1
r2
r3
6       →      timestamps[] size n = 6
5       →      timestamps = [5, 10, 18, 20, 40, 100]
10
18
20
40
100
```

Sample Output 0

```
2
```

Explanation

Request Retry Table

Request ID	Timestamps	Retry Pairs	Retry Count
r1	[10, 20]	(10, 20)	1
r2	[5, 18, 40]	(5, 18)	1
r3	[100]	-	0

The total number of retries = 1 + 1 + 0 = 2.

▼ **Sample Case 1**

Sample Input 1

```
STDIN          FUNCTION
-----          --------
5       →      gap = 5
5       →      requestIds[] size n = 5
x1      →      requestIds = ["x1", "x1", "x1", "x2", "x2"]
x1
x1
x2
x2
5       →      timestamps[] size n = 5
100     →      timestamps = [100, 103, 107, 200, 208]
103
107
200
208
```

Sample Output 1

```
2
```

Explanation

Request Retry Table

Request ID	Timestamps	Retry Pairs	Retry Count
x1	[100, 103, 107]	(100, 103), (103, 107)	2
x2	[200, 208]	-	0

The total number of retries = 2 + 0 = 2.

Question - 12
Trend Analytics Monitoring

You are building a system that analyzes performance metrics over time, where each trend is given by *trends[i]*. One feature involves identifying bitonic trends, i.e., sequences that first rise (non-decreasing) and then fall (non-increasing).

Implement a function to find the length of the longest subarray that is bitonic in nature.

The function *longestBitonicTrend* takes the following input:
int trends[n]: an array of integers

The function should return an integer, the length of the longest bitonic subarray.

Example
trends = [10, 8, 9, 15, 12, 6, 7]

[8, 9, 15, 12, 6] is the longest bitonic subarray. The non-decreasing subarray is [8, 9, 15], and the non-increasing portion is [15, 12, 6].

Hence, the answer is 5.

Constraints

- $1 \le n \le 10^5$
- $1 \le trends[i] \le 10^9$

▼ Input Format For Custom Testing

The first line contains an integer *n*, the number of elements in *trends*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *trends[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
6        →    trends[] size n = 6
5        →    trends = [5, 1, 2, 1, 4, 5]
1
2
1
4
5
```

Sample Output

```
3
```

Explanation
[1, 2, 1] is one of the longest bitonic subarrays. [1, 4, 5] is also bitonic and has the same length. It is non-decreasing through [1, 4, 5], and the non-increasing portion is [5].

Hence, the answer is 3.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
5        →    trends[] size n = 5
9        →    trends = [9, 7, 6, 2, 1]
7
6
2
1
```

Sample Output

```
5
```

Explanation
The longest bitonic subarray is [9, 7, 6, 2, 1]. The non-decreasing subarray is [9], and the non-increasing portion is the entire array.

Hence, the answer is 5.

Question - 13
Interaction Pair Retrieval

In a logging system, an API tracks every microservice interaction with the server. An array named *services* of length *n* exists, where *services[i]* represents the ID of the microservice interacting with the server at the ith moment.

To analyze interaction patterns, the system computes a cross-interaction log:

- The system generates all possible ordered service pairs (*services[i], services[j]*) for all $0 \le i, j < n$.
- These service interaction pairs are then sorted lexicographically.

Implement a function that returns the kth (1-based) interaction pair.

The function *getKthInteraction* takes the following inputs:
int services[n]: the list of service IDs
long k: the 1-based index of the interaction to retrieve

The function should return a list of two integers representing the kth lexicographically smallest interaction pair.

Note: An element *(a, b)* is lexicographically smaller than element *(x, y)* if it satisfies:

- $a < x$, or
- $a = x$ and $b < y$.

Example
$n = 3$
services = [3, 1, 2]
$k = 7$

All possible interactions = [(3,3), (3,1), (3,2), (1,3), (1,1), (1,2), (2,3), (2,1), (2,2)].
Lexicographically sorted order = [(1,1), (1,2), (1,3), (2,1), (2,2), (2,3), (3,1), (3,2), (3,3)].

The 7th interaction pair is [3, 1]. Hence, the function should return [3, 1].

Constraints

- $1 \le n \le 10^5$
- $1 \le services[i] \le 10^9$
- $1 \le k \le n^2$

▼ Input Format For Custom Testing

The first line contains an integer, *n*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *services[i]*.
The last line contains an integer, *k*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
2        →    services[] size n = 2
4        →    services = [4, 1]
1
3        →    k = 3
```

Sample Output

```
4
1
```

Explanation
All possible interactions = [(4, 4), (4, 1), (1, 4), (1, 1)]
Lexicographically sorted order = [(1, 1), (1, 4), (4, 1), (4, 4)].

The 3rd interaction pair is (4, 1). Hence, the function should return [4, 1].

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
3        →    services[] size n = 3
2        →    services = [2, 2, 1]
2
1
5        →    k = 5
```

Sample Output

```
2
1
```

Explanation

All possible interactions = [(2, 2), (2, 2), (2, 1), (2, 2), (2, 2), (2, 1), (1, 2), (1, 2), (1, 1)]

Lexicographically sorted order = [(1, 1), (1, 2), (1, 2), (2, 1), (2, 1), (2, 2), (2, 2), (2, 2), (2, 2)].

The 5th interaction pair is (2, 1). Hence, the function should return [2, 1].

Question - 14

Equivalence Checker

You are given two string arrays, *sourceNames* and *targetNames*, each containing *n* filenames. Your task is to determine whether each pair of corresponding filenames can be made identical through at most one bulk character replacement operation.

A bulk character replacement operation allows you to:

- Choose either the source filename or the target filename from a pair
- Select any single character in the chosen filename
- Replace all occurrences of that character with a different character

Implement a function that processes these filename pairs and returns the results.

The function *checkFilenameEquivalence* takes two parameters:

string sourceNames[n]: an array of source filenames

string targetNames[n]: an array of target filenames

The function should return an array of strings where each element corresponds to the result for a filename pair. Return "YES" if the pair can be made identical using the allowed operation, otherwise return "NO".

Example

n = 3

sourceNames = ["aaa", "abbc", "zyz"]

targetNames = ["bbb", "cccc", "zyx"]

Compare the filenames at each index:

- *i* = 0: *sourceNames[0]* = "aaa", *targetNames[0]* = "bbb"
  We can change all the occurrences of 'a' to 'b' in the first string in this case. Therefore, the answer is "YES".
- *i* = 1: *sourceNames[1]* = "abbc", *targetNames[1]* = "cccc"
  There is no way in which we can make these strings identical. Therefore, the answer is "NO".
- *i* = 2: *sourceNames[2]* = "zyz", *targetNames[2]* = "zyx"
  We can replace the only occurrence of 'x' in the second string with 'z' to make them identical. Therefore, the answer is "YES".

Hence, the answer corresponding to each pair of strings is ["YES", "NO", "YES"].

Constraints

- $1 \le n \le 10^3$
- $1 \le |sourceNames|, |targetNames| \le 10^3$, where $|s|$ denotes the length of string s
- All the strings consist of lowercase English letters only.

▼ Input Format For Custom Testing

The first line contains an integer *n*, the number of elements in *sourceNames*.
Each line *i* of the *n* subsequent lines (where *0 ≤ i < n*) contains a string *sourceNames[i]*.
The next line contains an integer *n*, the number of elements in *targetNames*.
Each line *i* of the *n* subsequent lines (where *0 ≤ i < n*) contains a string *targetNames[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
2        →    sourceNames[] size n = 2
ac       →    sourceNames = ["ac", "pqqrr"]
pqqrr
2        →    targetNames[] size n = 2
ca       →    targetNames = ["ca", "prrrr"]
prrrr
```

Sample Output

```
NO
YES
```

Explanation

- *i* = 0: *sourceNames[0]* = "ac", *targetNames[0]* = "ca"

  There is no way in which we can make these strings identical. Therefore, the answer is "NO".
- *i* = 1: *sourceNames[1]* = "pqqrr", *targetNames[1]* = "prrrr"

  We can replace all the occurrences of 'q' in the first string with 'r' to make them identical. Therefore, the answer is "YES".

Hence, the answer corresponding to each pair of strings is ["NO", "YES"].

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
2        →    sourceNames[] size n = 2
a        →    sourceNames = ["a", "aaabbb"]
aaabbb
2        →    targetNames[] size n = 2
b        →    targetNames = ["b", "abbbbb"]
abbbbb
```

Sample Output

```
YES
NO
```

Explanation

- *i* = 0: *sourceNames[0]* = "a", *targetNames[0]* = "b"

  We can change the only occurrence of 'a' to 'b' in the first string to make them identical. Therefore, the answer is "YES".
- *i* = 1: *sourceNames[1]* = "aaabbb", *targetNames[1]* = "abbbbb"

  There is no way in which we can make these strings identical. Therefore, the answer is "NO".

Hence, the answer corresponding to each pair of strings is ["YES", "NO"].

Question - 15
Minimize URL Compression

In a URL shortening service, the URL is given as a string *url* consisting of lowercase English letters.

The task is to extract a subsequence of exactly *k* characters from the *url* that minimizes the URL compression, defined as the difference between the length of the subsequence and the number of distinct characters in the subsequence.

Implement a function to compute the minimum possible compression for any subsequence of length *k*.

The function *minimizeURLCompression* takes the following inputs:
string url: a string representing the original URL code
int k: the desired length of the subsequence

The function should return the minimum possible compression in any subsequence of length *k*.

Note: The subsequence of a string is a string obtained by deleting any set of characters from the original string while retaining the order of remaining characters.

Example
url = "ooxoxo"
k = 4

One optimal subsequence of length exactly *k* is "ooxx". The length of the subsequence is 4, and it contains two distinct characters, 'o' and 'x'. The URL compression is 4 - 2 = 2.

Hence, the answer is 2.

Constraints
- $2 \le k \le \text{length}(url) \le 10^5$
- String *url* consists of lowercase English letters only.

▼ Input Format For Custom Testing

The first line contains a string, *url*.
The next line contains an integer *k*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
aaaabbbb  →    url = "aaaabbbb"
5         →    k = 5
```

Sample Output

```
3
```

Explanation

One optimal subsequence of length exactly *k* is "aabbb". It is 5 characters long and has two distinct characters, 'a' and 'b'. The compression is 5 - 2 = 3.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
abcdeabcde →   url = "abcdeabcde"
5          →   k = 5
```

Sample Output

```
0
```

Explanation

One optimal subsequence of length exactly *k* is "abcde". Its length is 5 and it has 5 distinct characters, 'a', 'b', 'c', 'd', and 'e'. Its compression is 5 - 5 = 0.

Question - 16

SEO Input Processing

You are working on a lightweight search engine indexing module, and one of the early tasks is to filter out non-informative words that appear too frequently, often called stop words. These words are so common across documents that they dilute search relevance.

Implement a function that finds all the words that occur in the string *text* at least *k* times.

The function *findStopWords* will take two inputs:

string text: a string containing words of letters in the English alphabet separated by space, " "
int k: the minimum number of occurrences of a word required to be considered as a stop word

The function should return an array of strings, the stop words in order of their first occurrence in the string.

Example

text = "a mouse is smaller than a dog but a dog is stronger"
k = 2

The list of stop words that occur at least *k* = 2 times is ["a", "is", "dog"]. "a" occurs 3 times, "is" and "dog" both occur 2 times. No other word occurs at least 2 times. The answer is in order of the first appearance in *text*.

Constraints

- *text* has at most 50000 characters.
- Every character in *text* is either an English lowercase letter or a space.
- *text* starts and ends with a letter. No two consecutive characters are spaces, i.e., *text* is a valid sentence.
- There will be at least one stop word in the text.
- $1 \le k \le 18$

▼ Input Format Format for Custom Testing

The first line contains the string *text*.
The second line contains the integer *k*.

▼ Sample Case 0

Sample Input 0

```
STDIN                                                              Function
-----                                                              --------
the brown fox jumps over the brown dog and runs away to a brown house → text
2                                                                  → k
```

Sample Output 0

```
the
brown
```

Explanation

"the" occurs 2 times and "brown" occurs 3 times. These words are returned in the order of their first occurrence in the text.

▼ Sample Case 1

Sample Input 1

```
STDIN                                                    Function
-----                                                    --------
foo bar foo baz foo → text
3                                                      → k
```

Sample Output 1

```
foo
```

Explanation

"foo" occurs 3 times.

Question - 17

Sum Pairs

Implement a function that counts the number of ordered pairs of 0-based indices *(x, y)* in an array *arr* of size *n* such that the sum *arr[x] + arr[y]* is greater than or equal to *lowerLimit* and less than or equal to *upperLimit*.

The function *getNumberOfPairs* will take three inputs:

int arr[n]: The given array
int lowerLimit: The lower limit of the sum
int upperLimit: The upper limit of the sum

The function should return a long integer, the number of pairs of indices *(x, y)* such that their sum *(arr[x] + arr[y])* is between *lowerLimit* and *upperLimit*, inclusive.

Note: An ordered pair is one in which the order of elements matters, that is, *(a,b)* is equal to *(u,v)* if and only if *a=u* and *b=v*.

Example

n = 4
arr = [4, 6, 8, 15]
lowerLimit = 18
upperLimit = 28

The pairs of 0-based indices with sum greater than or equal to 18 and less than or equal to 28 are (0, 3), (3, 0), (1, 3), (3, 1), (3, 2), and (2, 3).
Thus, the answer is 6.

Constraints

- $1 \le n \le 2*10^5$
- $-10^9 \le arr[i] \le 10^9$
- $-10^9 \le lowerLimit \le upperLimit \le 10^9$

▼ Input Format for Custom Testing

The first line contains an integer *n*, the size of the array arr.
Each of the next *n* lines contains an integer arr*[i]*.
The next line contains an integer *lowerLimit*.
The next line contains an integer *upperLimit*.

▾ Sample Case 0

Sample Input 0

```
STDIN         FUNCTION
-----         --------
4        →    arr[] size n = 4
5        →    arr = [5, 10, 15, 25]
10
15
25
21       →    lowerLimit = 21
29       →    upperLimit = 29
```

Sample Output 0

```
2
```

Explanation

The pairs of indices (0-based indexing) with a sum between 21 and 29 are (1, 2) and (2,1).

▾ Sample Case 1

Sample Input 1

```
STDIN         FUNCTION
-----         --------
4        →    arr[] size n = 4
8        →    arr = [8, 2, 6, 3]
2
6
3
6        →    lowerLimit = 6
12       →    upperLimit = 12
```

Sample Output 1

```
10
```

Explanation

All the ordered pairs of indices (0-based indexing) with sums between 6 and 12, inclusive, are: (0, 1), (0, 3), (1, 0), (1, 2), (2, 1), (2, 2), (2, 3), (3, 0), (3, 2) and (3, 3).

Question - 18

Search Query Matcher

In an e-commerce product search system, users input queries to search for items. You are given an array *products* of length *n* representing the product names and an array *queries* of length *q* containing search query strings.

Implement a function that, for each query string, returns all products that are anagrams of the query string. An anagram is any string that can be formed by rearranging the letters of another string.

The function *getProductMatches* takes the following inputs:

string products[n]: the list of all available product names

string queries[q]: the list of search query strings

The function should return a list of products for each query that are anagrams of the query string, sorted alphabetically.

Example

n = 4

products = ["duel", "speed", "dule", "cars"]

q = 2

queries = ["spede", "deul"]

- For *queries[0]* = "spede", the only anagram of "spede" is "speed". The answer for this query string, sorted alphabetically is ["speed"].
- For *queries[1]* = "deul", both the "duel" and "dule" are anagrams of "deul". The answer for this query string, sorted alphabetically is ["duel", "dule"].

Hence, the function should return [["speed"], ["duel", "dule"]].

Constraints

- 1 ≤ *n, q* ≤ 5000
- 1 ≤ length of *products[i]*, length of *queries[i]* ≤ 100
- It is guaranteed that each query word has at least one anagram in the *products* list.

▼ Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *products*.
Each of the next *n* lines contains a string, *products[i]*.
The next line contains an integer, *q*, the number of elements in *queries*.
Each of the next *q* lines contains a string, *queries[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
6        →     products[] size n = 6
allot    →     products = ["allot", "cat", "peach", "dusty", "act", "cheap"]
cat
peach
dusty
act
cheap
3        →     queries[] size q = 3
tac      →     queries = ["tac", "study", "peahc"]
study
peahc
```

Sample Output

```
act cat
dusty
cheap peach
```

Explanation

- For *queries[0]* = "tac", the anagrams of "tac" sorted in alphabetical order are "act" and "cat".
- For *queries[1]* = "study", the only anagram of "study" is "dusty".
- For *queries[2]* = "peahc", the anagrams of "peahc" sorted in alphabetical order are "cheap" and "peach".

Hence, the function should return [["act", "cat"], ["dusty"], ["cheap", "peach"]].

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
5        →     products[] size n = 5
emits    →     products = ["emits", "items", "baker", "times", "break"]
items
baker
times
break
2        →     queries[] size q = 2
mites    →     queries = ["mites", "brake"]
brake
```

Sample Output

```
emits items times
baker break
```

Explanation

- For *queries[0]* = "mites", the anagrams of "mites" sorted in alphabetical order are "emits", "items", and "times".
- For *queries[1]* = "brake", the anagrams of "brake" sorted in alphabetical order are "baker" and "break".

Question - 19

Algorithm Comparison

In a product recommendation system, two key-value pairs, *predictions1,* and *predictions2*, are given as strings. They represent the product recommendations made by two different algorithms.

Implement a function to find the keys where the recommended products (values) differ between the two algorithms. If a key is present in only one algorithm's recommendations, it should not be considered.

The function *getPredictionDiff* takes two inputs:
string *predictions1*: product recommendations made by Algorithm 1
string *predictions2*: product recommendations made by Algorithm 2

The function should return a list of the keys where the recommendations differ between the two algorithms, sorted alphabetically.

Example

predictions1 = '{"hello":"world","hi":"hello","you":"me"}'
predictions2 = '{"hello":"world","hi":"helloo","you":"me"}'

The only common key where the values differ is "hi". Hence the answer is ["hi"].

Constraints

- $1 \le |predictions1|, |predictions2| \le 10^5$. Every key is distinct in each of the predictions.
- There is no whitespace in the strings.

▼ Input Format For Custom Testing

The first line contains a string, *predictions1*.
The next line contains a string, *predictions2*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN                                          FUNCTION
-----                                          --------
{"hacker":"rank","input":"output"}       →     predictions1 = {"hacker":"rank","input":"output"}
{"hacker":"ranked","input":"wrong"}      →     predictions2 =
{"hacker":"ranked","input":"wrong"}
```

Sample Output

```
hacker
input
```

Explanation

Recommendations are different for both keys so they are returned in alphabetical order.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN                                          FUNCTION
-----                                          --------
{"hacker":"rank","input":"output"}     →       predictions1 = {"hacker":"rank","input":"output"}
{"hacker":"rank","input":"output"}     →       predictions2 = {"hacker":"rank","input":"output"}
```

Sample Output

Explanation

Recommendations match for both keys, so return an empty list.

Question - 20

Prediction Cache Retrieval

In a machine learning model, there are *n* data entries stored in the model's output cache *cacheEntries.* Each entry contains a *timestamp*, a *modelId*, and a *predictionValue*. The *timestamp* indicates when the prediction was made, *modelId* identifies the machine learning model, and *predictionValue* is the model's output as an integer represented as a string. Note that the *modelId* can repeat across entries.

The query handler receives *q* query strings in the array *queries[q][2]*, where each query is of the form {*modelId*, *timestamp*}. The *modelId* is the machine learning model's identifier, and *timestamp* represents the time the prediction was made.

Implement a function that determines the *predictionValue* for each query based on the given *modelId* and *timestamp*.

The function *getQueryAnswers* takes two inputs:

string cacheEntries[n][3]: the cache data entries.
string queries[q][2]: the queries.

The function should return an array of integers that represent the *predictionValue* for each query based on the given *modelId* and timestamp.

Example

n = 3
cacheEntries = [["12:30:22", "a2er5i80", "125"], ["09:07:47", "io09ju56", "341"], ["01:23:09", "a2er5i80", "764"]]
q = 2
queries = [["a2er5i80", "01:23:09"], ["io09ju56", "09:07:47"]]

- The *queries[0]* refers to the entry where the model a2er5i80 made a prediction at 01:23:09, with the *predictionValue* of 764.
- The *queries[1]* refers to the entry where the model io09ju56 made a prediction at 09:07:47, with the *predictionValue* of 341.

Event Type	Data	Cache	Query Result
Update cache	["09:07:47", "io09ju56", "341"]	io09ju56 = 341	-
Query	["io09ju56", "09:07:47"]	io09ju56 = 341	341
Update cache	["12:30:22", "a2er5i80", "125"]	io09ju56 = 341, a2er5i80 = 125	-
Update Cache	["01:23:09", "a2er5i80", "764"]	io09ju56 = 341, a2er5i80 = 764	-
Query	["a2er5i80", "01:23:09"]	io09ju56 = 341, a2er5i80 = 764	764

Return [764, 341].

Constraints

- $1 \le n \le 10^5$
- $1 \le q \le 10^5$
- $1 \le int(cacheEntries[i][2]) \le 10^8$

- *cacheEntries[i][0]* represents a valid timestamp in the format hh:mm: ss
- *size(cacheEntries[i][0])* = 8
- *cacheEntries[i][1],* is an alphanumeric value consisting of only lowercase English letters (a-z), and digits (0-9)
- It is guaranteed that the queried {*modelId*, *timestamp*} pair is present in the cache
- There are no duplicate timestamps for a particular *modelId.*

▼ Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *cacheEntries.*
The next line contains a constant integer, 3.
Each of the next *n* lines contains three space-separated strings, *cacheEntries[i][0], cacheEntries[i][1], cacheEntries[i][2].*
The next line contains a constant integer, *q*, the number of elements in *queries.*
The next line contains a constant integer, 2.
Each of the next *q* lines contains two space-separated strings, *query[i][0]* and *query[i][1].*

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN                          FUNCTION
-----                          --------
3                        →     cacheEntries [] size n = 3
3                        →     constant integer = 3 (elements per cacheEntries[i])
03:00:23 1e3fg08w 101    →     cacheEntries = [["03:00:23", "1e3fg08w", "101"],
12:45:36 ef41t56l 243                           ["12:45:36", "ef41t56l", "243"],
21:09:31 ko0e51j8 809                           ["21:09:31", "ko0e51j8", "809"]]
1                        →     queries[] size q = 1
2                        →     constant integer = 2 (elements per queries[i])
ef41t56l 12:45:36        →     queries = ["ef41t56l", "12:45:36"]
```

Sample Output

```
243
```

Explanation
The query corresponds to the prediction entry at index 1, with a value = 243.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN                          FUNCTION
-----                          --------
4                        →     cacheEntries[] size n = 4
3                        →     constant integer = 3 (elements per cacheEntries[i])
01:34:05 166k1kph 352    →     cacheEntries = [["01:34:05", "166k1kph", "352"],
56:38:37 8pvj20oo 107                           ["56:38:37", "8pvj20oo", "107"],
36:17:33 r0v06eec 180                           ["36:17:33", "r0v06eec", "180"],
20:34:20 e15y6dv4 490                           ["20:34:20", "e15y6dv4", "490"]]
2                        →     queries[] size q = 2
2                        →     constant integer = 2 (elements per queries[i])
e15y6dv4 20:34:20        →     queries = [["e15y6dv4", "20:34:20"],
8pvj20oo 56:38:37                            ["8pvj20oo", "56:38:37"]]
```

Sample Output

```
490
107
```

Explanation
The queries correspond to prediction entries at indices 3 and 1, respectively, with values 490 and 107.

Question - 21
Inventory Updates

A warehouse system keeps track of *numItems* different items with their quantities in an array *inventory*. Throughout the day, the warehouse receives *numUpdates* updates to the inventory. They can be of 2 different types:

1. *Item-Specific Update*:
   An update *[1, quantity, itemIndex]* updates the *itemIndex*th item (0-based index) to *quantity*.
2. *Bulk Update*:
   An update *[2, quantity, 0]* updates the quantities of all items in the inventory to *quantity*.

After each update, calculate the total inventory across all items in the warehouse. Return an array where the ith element is the total inventory after processing the ith update.

Note: The updates are persistent, meaning each one modifies the inventory based on its current state.

Example

numItems = 3
inventory = [1, 2, 3]
numUpdates = 3
updates = [[1, 2, 0], [2, 5, 0], [1, 2, 2]]

The array *inventory* is changed in the following way -

- *updates[0] = [1, 2, 0]*, update *inventory[0]* to 2: *inventory* = [2, 2, 3], sum = 7.
- *updates[1] = [2, 5, 0]*, update all items to 5: *inventory* = [5, 5, 5], sum = 15
- *updates[2] = [1, 2, 2]*, update *inventory[2]* to 2: *inventory* = [5, 5, 2], sum = 12.

Hence, the answer is [7, 15, 12].

Function Description

Complete the function *getSum* in the editor with the following parameters:

int inventory[numItems]: the array *inventory*
int updates[numUpdates][3]: the given *updates*

Returns

long[numUpdates]: the sum of all the elements of the array after each update.

Constraints

- $1 \le numItems, numUpdates \le 2 * 10^5$
- $1 \le inventory[i], updates[i][1] \le 10^9$
- *updates[i][0]* is either 1 or 2.
- $0 \le updates[i][2] \le n-1$

▼ Input Format for Custom Testing

The first line contains an integer *numItems*, the size of array *inventory.*
Each of the next *numItems* lines contains an integer *inventory[i].*
The next line contains an integer *numUpdates*, the number of *updates*.
Each of the next *numUpdates* lines contains three integers, *updates[i][0], updates[i][1]* and *updates[i][2]* respectively.

▼ Sample Case 0

Sample Input 0

```
STDIN         Function
-----         --------
3        →    inventory[] size numItems = 3
10       →    inventory = [10, 6, 7]
6
7
2        →    updates[] size numUpdates = 2
3        →    updates[][] size = 3
```

```
1 7 0     →    updates = [[1, 7, 0], [1, 7, 1]]
1 7 1
```

Sample Output 0

```
20
21
```

Explanation

The array *inventory* is changed in the following way -

- After *updates[0]*, *inventory* = [7, 6, 7], sum = 20.
- After *updates[1]*, *inventory* = [7, 7, 7], sum = 21

Hence, the answer is [20, 21].

▼ Sample Case 1

Sample Input 1

```
STDIN         Function
-----         --------
4        →    inventory[] size numItems = 4
1        →    inventory = [1, 3, 7, 1]
3
7
1
2        →    updates[] size numUpdates = 2
3        →    updates[][] size = 3
2 5 0    →    updates = [[2, 5, 0], [1, 5, 0]]
1 5 0
```

Sample Output 1

```
20
20
```

Explanation

The array *inventory* is changed in the following way -

- After *updates[0]*, *inventory* = [5, 5, 5, 5], sum = 20.
- After *updates[1]*, *inventory* = [5, 5, 5, 5], sum = 20.

Hence, the answer is [20, 20].

Question - 22

Folder Compression

You have a folder with *n* files of different sizes. The folder is compressed when files are arranged in non-decreasing order of size.

The operation takes one second to complete. You can perform this operation any number of times:

1. Extract the first file in the *fileSize* array and insert it at the back.
2. Swap the file with the one before it until it reaches either the front or a position where its size is strictly greater than the file in front.

Return the minimum number of seconds required to compress the folder, or -1 if it is impossible.

Example:

fileSize = [4, 3, 1, 3, 5]

Operations:

1. Remove file with size 4 and insert it at the back: [3, 1, 3, 5, **4**]; swap 4 once: [3, 1, 3, **4**, 5]
2. Remove file with size 3 and insert it at the back: [1, 3, 4, 5, **3**]; swap 3 into correct position: [1, **3**, 3, 4, 5]

The folder is compressed after 2 seconds.

Function Description

Complete the function *getMinimumSeconds* in the editor with the following parameters

int fileSize[n]: the sizes of the files

Returns

int: the number of seconds required to compress the folder or -1 if impossible.

Constraints

- $1 \le n \le 2 * 10^5$
- $1 \le fileSize[i] \le 10^9$

▾ Input Format for Custom Testing

The first line contains an integer *n*, the number of files.

Each of the next *n* lines contains an integer *fileSize[i]*.

▾ Sample Case 0

Sample Input 0

```
STDIN    Function
-----    --------
5     → fileSize[] n = 5
4    →  fileSize = [4, 2, 3, 1, 2]
2
3
1
2
```

Sample Output 0

```
3
```

Explanation

Move 4 to the back of the list: [2, 3, 1, 2, **4**]

Move 2 to the back of the list: [3, 1, 2, 4, **2**] and shift it left: [3, 1, **2**, 2, 4]

Move 3 to the back of the list: [1, 2, 2, 4, **3**] and shift it left: [1, 2, 2, **3**, 4]

The list is now sorted.

▾ Sample Case 1

Sample Input 1

```
STDIN    Function
-----    --------
5     →  fileSize[] n = 5
5     →  fileSize = [5, 3, 2, 3, 2]
3
2
3
2
```

Sample Output 1

```
-1
```

Explanation

Move 5 to the back of the list: [3, 2, 3, 2, **5**] and shift it left: [**5**, 3, 2, 3, 2].

There is no way to arrange files in order of non-decreasing sizes using the given operation.

Question - 23

Neural Task Allocation

You are given a set of tasks and a neural network system with multiple nodes. Each node can process tasks efficiently, but for optimal performance, tasks must be distributed evenly across all nodes. Your goal is to maximize the total number of tasks completed.

Each node must:

- Process exactly the same number of tasks as every other node
- Never process two tasks of the same type

Determine the maximum number of tasks that can be completed when distributed optimally across the neural network.

Example

There are $n = 7$ tasks

task = [1, 2, 2, 1, 3, 1, 3]

nodeCount = 2

One optimal allocation:

- Node 1 processes tasks at positions 1, 2, and 5, 1-based indexing, (values 1, 2, and 3)
- Node 2 processes tasks at positions 3, 4, and 7 (values 2, 1, and 3)

This allows 6 tasks to be completed in total. Each node processes 3 tasks, and no node processes two tasks of the same type.

Function Description

Complete the function *findMaximumTasks* in the editor with the following parameter(s):

int *task[n]*: the type of each task

int *nodeCount:* the number of nodes in the system

Returns

int: the maximum number of tasks that can be completed by distributing tasks optimally

Constraints

- $1 \le n \le 3 * 10^5$
- $1 \le task[i] \le 10^9$
- $1 \le nodeCount \le 10^9$

▼ Input Format For Custom Testing

The first line contains an integer *n*, the number of elements in *task*.

Each of the next *n* lines contains an integer *task[i]*.

The next line contains an integer *nodeCount*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN           FUNCTION
-----           --------
4         →     task[] size n = 4
1         →     task = [1, 2, 1, 1]
2
1
1
2         →     nodeCount = 2
```

Sample Output

```
2
```

Explanation

The first node works on the first task, and the second node works on the second task.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN           FUNCTION
-----           --------
3         →     task[] size n = 3
1         →     task = [1, 3, 2]
3
2
3         →     nodeCount = 3
```

Sample Output

```
3
```

Explanation

The first node works on the first task, the second node works on the second task, and the third node, on the third task.

Question - 24

Bandwidth Allocation

A service provider manages *n* API endpoints, each serving different functionalities, numbered from 1 to *n*. Given *m* bandwidth units, allocate these among the endpoints.

The objective is to maximize the bandwidth allocated to the kth endpoint while meeting the following conditions:

- All bandwidth units must be allocated.
- Each endpoint must have at least 1 unit of bandwidth allocated.
- The absolute difference in bandwidth between any two adjacent endpoints cannot exceed 1.

Return the maximum possible bandwidth that can be allocated to the kth endpoint while satisfying all conditions.

Example

$n = 4$

$k = 3$

$m = 17$

One optimal way to distribute $m = 17$ units of bandwidth among the $n = 4$ endpoints is 3, 4, 5, 5.
Here, the 3rd endpoint is allocated 5 units of bandwidth. Hence, 5 is the required answer.

Function Description

Complete the function *findMaximumBandwidths* in the editor with the following parameter(s):

int n: the number of API endpoints
int k: the endpoint number for which bandwidth is to be maximized
int m: the total bandwidth to distribute among the endpoints

Returns

int: the maximum units of bandwidth that can be allocated to the kth endpoint

Constraints

- $1 \le n \le 10^9$
- $1 \le k \le n$
- $n \le m \le 10^9$

▼ Input Format For Custom Testing

The first line contains an integer *n*, the number of endpoints.
The second line contains an integer *k,* the endpoint to optimize bandwidth for.
The third line contains an integer *m*, the bandwidth to distribute.

▼ **Sample Case 0**

Sample Input For Custom Testing

```
STDIN        FUNCTION
------       ---------
6        →   n = 6
2        →   k = 2
11       →   m = 11
```

Sample Output

```
3
```

Explanation

One optimal way to distribute 11 units of bandwidth among the 6 endpoints is 3, 3, 2, 1, 1, 1.
Here, the 2nd endpoint is allocated 3 bandwidth allocations.

▼ **Sample Case 1**

Sample Input For Custom Testing

```
STDIN        FUNCTION
------       ----------
5       →    n = 5
3       →    k = 3
14      →    m = 14
```

Sample Output

```
4
```

Explanation

One way to distribute the bandwidth among the 5 endpoints is: 2, 3, 4, 3, 2.
Here, the 3rd endpoint is allocated 4 bandwidth allocations.

Question - 25

Endpoint Inspection

Analyze a *BandwidthOptimizer* system that inspects API endpoints for bandwidth issues. The system operates as follows:

- There are *m* API endpoints numbered 1 to *m*.
- *n* specific endpoints in the *endpoint* array have confirmed bandwidth issues.
- The optimizer takes one second to reach endpoint 1 and one second for each transition. It traverses the endpoints from 1 to *m*.
- The system inspects endpoints when either:
  - There is a known issue at that endpoint.
  - At least *k* seconds have passed since the last inspection.

Your task is to determine the minimum number of inspections required if one problematic endpoint is manually corrected, and how many endpoints achieve this minimum.

Example

m = 5
n = 2
k = 2
endpoint = [2, 5]

There are 5 endpoints with 2 issues at endpoints 2 and 5. The interval between checks is 2 seconds.

- If the error at endpoint 2 is manually corrected, the optimizer will check points 2, 4, and 5.
  - The error is manually corrected at endpoint 2.
  - Traversal starts. Two-second intervals occur at positions 2 and 4, so they are checked.

    - Endpoint 5 is known to have issues, so it must be checked.
- If the error at endpoint 5 is manually corrected, the optimizer will check points 2 and 4.
    - The error is manually corrected at endpoint 5.
    - Traversal starts. Two-second intervals occur at positions 2 and 4, so they are checked.

So, the minimum number of checks is two for the case when the issue at endpoint 5 is manually corrected. Only one scenario requires just two checks.

Hence, the answer is 2 checks and 1 scenario, i.e., [2, 1].

Function Description

Complete the function *getMinChecks* in the editor with the following parameter(s):

int endpoint[n]: the API endpoints with known bandwidth issues

int m: the total number of API endpoints

int k: the time that the optimizer must wait before checking an endpoint for bandwidth issues

Returns

int[2]: the integer at the first index represents the minimum number of endpoints that must be checked after resolving the issues at one endpoint, and the second integer represents the number of endpoints where this minimum number of inspections is attained

Constraints

- $1 \le k \le m \le 10^9$
- $1 \le n \le \min(2 * 10^5, m)$
- $1 \le endpoint[i] \le m$

▼ Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *endpoint*.

Each of the *n* lines contains an integer, *endpoint[i]*.

The next line contains an integer *m*.

The next line contains an integer *k*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN           FUNCTION
------          -------------
3        →      endpoint[] size n = 3
3        →      endpoint = [3, 5, 8]
5
8
8        →      m = 8
2        →      k = 2
```

Sample Output

```
4
1
```

Explanation

- If the error at endpoint 3 is corrected manually, the optimizer will check points 2, 4, 5, 7, and 8.
- If the error at endpoint 5 is corrected, the optimizer will check points 2, 3, 5, 7, and 8.
- If the error at endpoint 8 is corrected, the optimizer will check points 2, 3, 5, and 7

So, minimum checks = 4, if the error at position 8 is manually corrected.

Hence, the answer is [4, 1].

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN           FUNCTION
------          -------------
4          →    endpoint[] size n = 3
```

```
8          →   endpoint = [8, 2, 9, 10]
2
9
10
10         →   m = 10
9          →   k = 9
```

Sample Output

```
3
4
```

Explanation

- If the error at endpoint 8 is corrected, the optimizer will check points 2, 9, and 10.
- If the error at endpoint 2 is corrected manually, the optimizer will check points 8, 9, and 10.
- If the error at endpoint 9 is corrected, the optimizer will check points 2, 8, and 10.
- If the error at endpoint 10 is corrected, the optimizer will check points 2, 8, and 9.

So, minimum checks = 3, for all the points.

Hence, the answer is [3, 4].

Question - 26

Workload Optimization

In a software development team of *n* developers, each day, *n - 1* developers resolve issues while one documents the resolution. You are given an array *developerData* where each element represents the number of days a developer needs to handle issues. The goal is to determine the minimum number of days required for all developers to meet their targets through optimized task assignments.

Example

n = 4

developerData = [1, 4, 3, 3]

An optimal strategy that minimizes the work duration:

- For the first three days, Developer 1 documents while the others handle issues, resulting in [0, 3, 3, 3].
- On the fourth day, Developer 3 takes over documentation, resulting in [1, 4, 3, 4].

The minimum number of days required is 4.

Function Description

Complete the function *getMinimumDays* in the editor with the following parameter(s):

int developerData[n]: the minimum number of days each developer must handle issues

Returns

int: the minimum number of days needed for every developer to complete their responsibilities

Constraints

- $2 \le n \le 2 * 10^5$
- $1 \le developerData[i] \le 10^9$

▼ Input Format for Custom Testing

The first line contains an integer, *n*, the number of developers.

Each of the next *n* lines contains an integer *developerData[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
```

```
5          →   developerData[] size n = 5
2          →   developerData = [2, 1, 3, 4, 5]
1
3
4
5
```

Sample Output

```
5
```

Explanation

One optimal strategy is:

- On the first two days, the second developer documents while others handle issues resulting in [2, 0, 2, 2, 2].
- The first developer documents for the next three days resulting in [2, 3, 5, 5, 5].

All developers have finished on the fifth day.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
6          →   developerData[] size n = 6
1          →   developerData = [1, 1, 2, 1, 3, 3]
1
2
1
3
3
```

Sample Output

```
3
```

Explanation

One optimal strategy is:

- The first developer documents for the first two days: [0, 2, 2, 2, 2, 2].
- The second developer documents on the next day while others handle issues: [1, 2, 3, 3, 3, 3].

Developers complete their tasks on the third day.

Question - 27

Model Testing

When evaluating a machine learning model, you are given *n* test cases, each associated with an *arrivalTime[i]* indicating when each test case was submitted. The testing environment can be activated for a certain period and then deactivated. If activated at time *t1* and deactivated at time *t2*, all tests with arrival times between *t1* and *t2* (inclusive) are executed. The total active duration of the system is *t2 – t1*.

The efficiency of the testing system is defined as the number of test cases executed minus the total active time of the system.

Your task is to determine the maximum efficiency by selecting the optimal activation and deactivation times. The environment must execute at least two test cases during its active period. Return the maximum possible efficiency.

Notes:

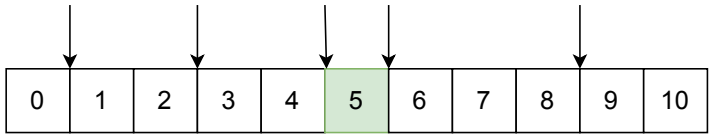
- Execution time for test cases is negligible.
- If two test cases have the same arrival time, they are evaluated simultaneously.
- Efficiency can be negative.

Example

n = 5

arrivalTime = [9, 1, 3, 5, 6]

It is optimal to choose *t1* = 5 and *t2* = 6. If the testing environment is active from *t* = 5 to 6, the 4th and 5th test cases will be executed. The number of test cases tested = 2 and the time the system was active is 6 - 5 = 1. The efficiency of the system is 2 - 1 = 1.



Return 1 as the answer.

Function Description

Complete the function *getMaxEfficiency* in the editor with the following parameter(s):

int arrivalTime[n]: the arrival time of each prompt

Returns

int: the maximum efficiency possible

Constraints

- $2 \le n \le 2 * 10^5$
- $1 \le arrivalTime[i] \le 10^9$

▼ Input Format For Custom Testing

The first line contains an integer *n*, the number of elements in *arrivalTime*.

Each of the next *n* lines contains an integer *arrivalTime[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN           FUNCTION
------          -------------
3          →    arrivalTime[] size n = 3
4          →    arrivalTime = [4, 2, 1]
2
1
```

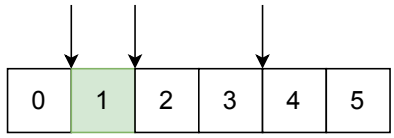
Sample Output

```
1
```

Explanation

Choose *t1* = 1 and *t2* = 2.

If the system is active from *t* = 1 to 2, test cases 1 and 2 will be tested.



The efficiency of the system is 2 - 1 = 1.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN           FUNCTION
------          -------------
7          →    arrivalTime[] size n = 7
8          →    arrivalTime = [8, 4, 6, 2, 7, 6, 6]
4
```

```
6
2
7
6
6
```

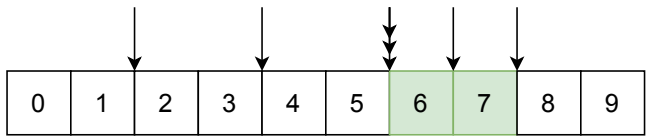
Sample Output

```
3
```

Explanation

Choose *t1* = 6 and *t2* = 8.

If the testing system is active from *t* = 6 to 8, test cases 0, 2, 4, 5, and 6 will be tested.



The efficiency of the system in this case is 5 - 2 = 3.

Question - 28

Dynamic Resource Allocation

A company needs an efficient strategy to optimize resource allocation for servers based on performance metrics. The performance of each server is represented by *performance[i]*, where 0 ≤ *i* < n. A server's rank is defined by the count of distinct performance metrics that are greater than or equal to its own performance metric.

To allocate resources, two positive integer constants, *inc,* and *dec*, are used to determine increment and decrement values, respectively. The resource allocation for each server is calculated as follows:

$$\text{resource allocation} = \text{inc} \times (n + 1 - \text{server's rank}) - \text{dec} \times (\text{total number of servers with the same rank as the server})$$

The task is to calculate the resource allocation for each server based on their performance and ranking.

Example

inc = 500

dec = 100

n = 6

performance = [3, 2, 4, 3, 5, 5]

Server Index 0 has a rank of 3 since 3, 4, and 5 are greater than or equal to its performance of 3. The resource allocation is calculated as 500 × (6 + 1 - 3) - 100 × 2 = 1800.

The resource allocation for the servers is [1800, 1400, 2400, 1800, 2800, 2800].

Function Description

Complete the function *getResources* that takes the following parameters:

int inc: the increment value

int dec: the decrement value

int performance[n]: an array indicating the performances of the servers

Returns

int[n]: an array of integers representing the resources allocated to each server

Constraints

- 1 ≤ *inc, dec* ≤ 10^4

- $1 \le n \le 10^5$
- $1 \le performance[i] \le 10^9$

▼ Input Format For Custom Testing

The first line contains an integer *inc*.
The second line contains an integer *dec*.
The third line contains an integer *n*, the number of elements in *performance*.
Each of the next *n* lines contains an integer *performance[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN           FUNCTION
-----           --------
200       →     inc = 200
50        →     dec = 50
5         →     performance[] size n = 5
2         →     performance = [2, 1, 3, 3, 1]
1
3
3
1
```

Sample Output

```
750
500
900
900
500
```

Explanation

Calculating rank and resource allocation of Servers

Server Index	Rank of each server (Distinct Performances greater than or equal to server's performance)	Resources allocated to each server
0	2 (2, 3)	200 * (5 + 1 - 2) - 50 * 1 = 750
1	3 (1, 2, 3)	200 * (5 + 1 - 3) - 50 * 2 = 500
2	1 (3)	200 * (5 + 1 - 1) - 50 * 2 = 900
3	1 (3)	200 * (5 + 1 - 1) - 50 * 2 = 900
4	3 (1, 2, 3)	200 * (5 + 1 - 3) - 50 * 2 = 500

The resources allocated to the servers = [750, 500, 900, 900, 500].

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN           FUNCTION
-----           --------
400       →     inc = 400
200       →     dec = 200
6         →     performance[] size n = 6
1         →     performance = [1, 2, 2, 2, 5, 7]
2
2
2
```

```
5
7
```

Sample Output

```
1000
1000
1000
1000
1800
2200
```

Explanation

Calculating rank and resource allocation of Servers

Server Index	Rank of each server (Distinct Performances greater than or equal to server's performance)	Resources allocated to each server
0	4 (1, 2, 5, 7)	400 * (6 + 1 - 4) - 200 * 1 = 1000
1	3 (2, 5, 7)	400 * (6 + 1 - 3) - 200 * 3 = 1000
2	3 (2, 5, 7)	400 * (6 + 1 - 3) - 200 * 3 = 1000
3	3 (2, 5, 7)	400 * (6 + 1 - 3) - 200 * 3 = 1000
4	2 (5, 7)	400 * (6 + 1 - 2) - 200 * 1 = 1800
5	1 (7)	400 * (6 + 1 - 1) - 200 * 1 = 2200

The resources allocated to the servers = [1000, 1000, 1000, 1000, 1800, 2200]

Question - 29
Cache Queries

Implement a prototype of a simple cache query handler that manages data entries stored in a cache.

Each cache entry has the format:

{timestamp, key, value}

- *timestamp*: when the entry was stored, in "hh:mm:ss" format
- *key*: an alphanumeric ID (e.g., "a2er5i80")
- *value*: the data value, an integer stored as a string (e.g., "125")

The handler receives queries of the form:

{key, timestamp}

For each query, you must return the corresponding data value stored in the cache for that exact {key, timestamp} pair.

Note:

- The same key can appear multiple times in the cache with different timestamps.
- For a given *timestamp*, a *key* is unique (no duplicate *{timestamp, key}* pairs).
- It is guaranteed that every queried *{key, timestamp}* exists in the cache.

You are given:

- cache_entries: a 2D array of strings, where each entry is *[timestamp, key, value]*
- queries: a 2D array of strings, where each query is *[key, timestamp]*

Return an array of integers representing the data values (converted from string to int) for each query.

Example

Suppose *cache_entries* = [["12:30:22", "a2er5i80", "125"], ["09:07:47", "io09ju56", "341"], ["01:23:09", "a2er5i80", "764"]] and *queries* = [["a2er5i80", "01:23:09"], ["io09ju56", "09:07:47"]]

Output: [341, 764]

Processing sequence:

1. Update cache with ["09:07:47", "io09ju56", "341"] → io09ju56 = 341
2. Query ["io09ju56", "09:07:47"] → Result: 341
3. Update cache with ["12:30:22", "a2er5i80", "125"] → io09ju56 = 341, a2er5i80 = 125
4. Update cache with ["01:23:09", "a2er5i80", "764"] → io09ju56 = 341, a2er5i80 = 764
5. Query ["a2er5i80", "01:23:09"] → Result: 764

Constraints

- $1 \leq$ size of *cache_entries[][2]* $\leq 10^5$
- $1 \leq$ size of *queries[][2]* $\leq 10^5$
- $1 \leq$ *int(cache_entries[i][2])* $\leq 10^8$
- *cache_entries[i][0]* represents a valid timestamp in the format hh:mm:ss
- size of *cache_entries[i][0]* = 8
- *cache_entries[i][1],* is an alphanumeric value, consisting of only lowercase English letters (a-z), and digits (0-9)
- It is guaranteed that the queried {key, timestamp} pair is present in the cache.
- At a particular timestamp, there can be no duplicate keys.

Test Case Input Format

The first line contains an integer, *n*, the number of elements in *cache_entries.*

The next line contains a constant integer, 3.

Each of the next *n* lines contains three space-separated strings, *cache_entries[i][0], cache_entries[i][1], cache_entries[i][2].*

The next line contains a constant integer, *q*, the number of elements in *queries.*

The next line contains a constant integer, 2.

Each of the next *q* lines contains two space-separated strings, *queries[i][0]* and *queries[i][1].*

Question - 30

Word Count Tool

Identify the number of valid words in a given string. A word is considered valid if it meets all of the following criteria:

- Contains at least 3 characters
- Contains only alphanumeric characters (letters 'a'-'z', 'A'-'Z', and numbers 0-9)
- Includes at least one vowel ('a', 'e', 'i', 'o', 'u')
- Includes at least one consonant (any letter that is not a vowel)

Return the count of valid words in the input string.

Example 1

Suppose *s* = "This is an example string 234".

Output: 3

Word	Is Valid	Reason
This	Yes	At least 3 characters, contains a vowel and a consonant
is	No	Less than 3 characters
an	No	Less than 3 characters
example	Yes	At least 3 characters, contains a vowel and a consonant
string	Yes	At least 3 characters, contains a vowel and a consonant
234	No	Does not contain a vowel or a consonant

Example 2

Suppose *s* = "This is Form16 submis$ion date".

Output: 3

Word	Is Valid	Reason
This	Yes	At least 3 characters, contains a vowel and a consonant
is	No	Less than 3 characters
Form16	Yes	At least 3 alphanumeric characters, contains a vowel and a consonant
submis$ion	No	Contains an invalid character
date	Yes	At least 3 characters, contains a vowel and a consonant

Constraints

- $1 \leq$ length of $s \leq 10^5$
- *s* consists of all available ASCII characters.

Question - 31

Cloud Data Transfer

In order to shift its infrastructure to the cloud, a company plans to rent *k* computing cores per day over *n* days. They have *m* pricing plans available for this period. Each pricing plan includes:

- *l[i]* and *r[i]*: the starting and ending days the plan is available
- *c[i]*: the number of cores available each day
- *p[i]*: the cost to rent one core per day

The company can rent between 0 and *c[i]* cores each day from each available plan. Given *n, k*, and the list of plans as a 2D array, return the minimum cost to rent the required cores.

Example

Suppose *n* = 5, *k* = 7, and *plans* = [[1, 3, 5, 2], [1, 4, 5, 3], [2, 5, 10, 1]].

The optimal strategy is shown.

Day	Purchase cores/plan	Cost

1	5 from plan 1, 2 from plan 2	5*2 + 2*3 = 16
2	7 from plan 3	7*1 = 7
3	7 from plan 3	7*1 = 7
4	7 from plan 3	7*1 = 7
5	7 from plan 3	7*1 = 7

The sum of costs is 16+7+7+7+7 = 44. Return 44.

Function Description

Complete the function *getMinCost* in the editor with the following parameter(s):

int *n* : the number of days to rent cores

int *k* : the number of cores to rent each day

int *plans[m][4]:* each *plan[i]* contains *[l[i], r[i], c[i], p[i]],* the start and end days, cores available, and price per core per day

Returns

long: the minimum cost to complete the process

Constraints

- $1 \le n, m \le 10^4$
- $1 \le k \le 500$
- $1 \le plans[0], plans[1] \le n$
- $1 \le plans[2] \le k$
- $1 \le plans[3] \le 10^9$
- It is guaranteed that for any available day, there are at least *k* cores available.

▼ Input Format For Custom Testing

The first line contains an integer, *n*.

The next line contains an integer, *k*.

The next line contains an integer, *m*.

The next line contains an integer, *4*, the number of columns in each *plans[i]* array.

Each line *i* of the subsequent m lines (where $0 \le i < m$) contains 4 space-separated integers that describe the ith plan.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN           FUNCTION
-----           --------
4          →    n = 4
4          →    k = 4
4          →    plans[] size m = 4
4          →    plans[][] size const = 4
1 4 4 5    →    plans = [[1, 4, 4, 5], [1, 4, 2, 5], [1, 2, 2, 1], [2, 3, 3, 2]]
1 4 2 5
1 2 2 1
2 3 3 2
```

Sample Output

```
49
```

Explanation

Day	Purchase cores/plan	Cost
1	2 from plan 1, 2 from plan 3	2*5 + 2*1 = 12
2	2 from plan 3, 2 from plan 4	2*1 + 2*2 = 6
3	1 from plan 2, 3 from plan 4	1*5 + 3*2 = 11
4	4 from plan 1	4*5 = 20

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
1         →    n = 1
4         →    k = 4
1         →    plans[] size m = 1
4         →    plans[][] size const = 4
1 1 4 5   →    plans = [ [1, 1, 4, 5] ]
```

Sample Output

```
20
```

Explanation

On day 1, choose 4 cores from plan 1, 4*5 = 20.

Question - 32
LRU Cache

Given an array of *n* request IDs as strings and an integer *k*, determine the *k* most recent unique requests after all requests have been received. The results should be ordered from the most recent to the least recent.

Example:

Suppose *n = 5, requests* = ["item1", "item2", "item3", "item1", "item3"], and *k* = 3.

Output: ["item3", "item1", "item2"]

Explanation:

Starting from the right and moving left, collecting distinct *requests*, there is "item3" followed by "item1". Skip the second instance of "item3" and find "item2".

Constraints

- $1 \le k \le n \le 10^5$
- *requests[i]* consists of lowercase letters and digits only, [a-z,0-9].

Question - 33
Autocorrect Prototype

Implement an autocorrect function that returns all words which are anagrams of a search query. An anagram is any string that has the same characters in the same frequency as another string.

Given two arrays, *words* and *queries* of length *n* and *q*, respectively, for each query, return an array of strings that are anagrams of the query, sorted in alphabetical order.

Example

Suppose *words* = ["duel", "speed", "dule", "cars"] and *queries* = ["spede", "deul"].

Output: [["speed"], ["duel", "dule"]].

Explanation:

- For "spede", the only anagram is "speed"
- For "deul", the anagrams are "duel" and "dule"

Constraints

- $1 \le n, q \le 5000$
- $1 \le$ length of *words*[i], length of *queries*[i] ≤ 100
- It is guaranteed that each query word has at least one anagram in *words*.
- All strings consist of lowercase English letters only.

Question - 34

Matrix Transformation

An array *arr* of *n* integers is transformed into a square matrix of size *n x n* as follows.

- If $i \ne j$, the value of the cell at the ith row and j^{th} column is $2 \times arr[i] + arr[j]$.
- If $i = j$, the value is 0.

Given a matrix *mat* of size *n x n,* report an array whose transformation will result in the given matrix.

Example

n = 2

mat=[[0, 3], [3, 0]]

The only array that can be transformed to the given matrix is [1, 1]. The transformations are:

i	j	value	comments
0	0	0	$i = j \rightarrow 0$
0	1	3	$i \ne j \rightarrow 2 * 1 + 1 = 3$
1	0	3	$i \ne j \rightarrow 2 * 1 + 1 = 3$
1	1	0	$i = j \rightarrow 0$

Hence, the answer is [1, 1].

Function Description

Complete the function *getOriginalArray* in the editor with the following parameters:

int mat[n][n]: a 2-dimensional matrix of integers

Returns

int[]: the array that can be transformed to form the matrix *mat*

Constraints

- $2 \le n \le 10^3$
- $1 \le mat[i][j] \le 3 * 10^5$

- *mat[i][i]* = 0, for 0 ≤ i < *n*
- There is always a unique array that can be transformed into the given matrix.

▼ Input Format For Custom Testing

The first line contains an integer, *n*, the number of rows in the matrix *mat*.
The second line contains the same integer, *n*, the number of columns in the matrix *mat*.

Each line *i* of the *n* subsequent lines (where 0 ≤ *i* < *n*) contains *n* space-separated integers, each a value of *mat[i][j],* i.e., the value of the cell in row *i* and column *j*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
2        →     mat[] size n = 2
2        →     mat[][] size n = 2
0 4      →     mat = [[0, 4], [5, 0]]
5 0
```

Sample Output

```
1
2
```

Explanation
The only array that can be transformed to the given matrix is [1, 2]. The two non-zero elements at (1, 0) and (0, 1) are generated using the transformation formula 2 * 2 + 1 = 5 and 2 * 1 + 2 = 4 respectively.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
3        →     mat[] size n = 3
3        →     mat[][] size n = 3
0 9 10   →     mat = [[0, 9, 10], [6, 0, 4], [8, 5, 0]]
6 0 4
8 5 0
```

Sample Output

```
4
1
2
```

Explanation
The only array that can be transformed to the given values is [4, 1, 2].

column 0	column 1	column 2
0	2 * 4 + 1 = 9	2 * 4 + 2 = 10
2 * 1 + 4 = 6	0	2 * 1 + 2 = 4
2 * 2 + 4 = 8	2 * 2 + 1 = 5	0

Question - 35
Efficient Teams

A coding competition wants to form teams of exactly two participants.

Each participant has a skill value given in the array skill.

You must pair them up so that:

- Every participant belongs to exactly one team.
- All teams have the same sum of skills.
  That is, for every pair *[a, b]*, *a + b* is the same value.

The efficiency of a team is defined as the product of the skills of its members:

- For a team *[a, b]*, efficiency = *a * b*.

You need to:

- Form valid teams (if possible),
- Compute the sum of efficiencies of all teams,
- If it is not possible to pair everyone such that all pairs have the same sum, return -1.

You are told that, if a valid arrangement exists, the answer is unique.

Example 1
Suppose the skills of the candidates are *skill* = [1, 2, 3, 2].
Output: 7

Explanation:
They can be paired as [[1, 3], [2, 2]]. The sum of skills for each team is the same, i.e., 4.
The efficiency is computed as:

- Efficiency of [1, 3] = 1 * 3 = 3
- Efficiency of [2, 2] = 2 * 2 = 4

The sum of efficiencies is 3 + 4 = 7.

Example 2
Suppose the skills of the candidates are *skill* = [2, 1, 1, 4, 3, 5].
Output: -1

Explanation:
There is no way to form 3 teams such that the sum of skills of each pair is the same.

Constraints

- $1 \le n \le 10^5$
- $1 \le skill[i] \le 10^5$
- *n* is even.
- It is guaranteed that the answer is unique.

Question - 36
The Organized Shop

A row of items is considered organized if the parity (even or odd) of each stack of items is different from its adjacent stacks. In other words, if one stack has an even number of items, the stacks next to it must have odd numbers of items, and vice versa.

To organize a row, you can perform operations where half of the items in any stack are removed. You can perform this operation as many times as needed on any stack.

Your task is to determine the minimum number of operations required to organize the row.

Formally, given an array *items[]* of integers of length *n*, the array is organized if for each *x* less than *n-1*, *items[x]* mod 2 != *items[x+1]* mod 2. In one operation, you can choose an element at index *x* and set *items[x] = floor(items[x]/2)*.

Example 1
Suppose *items* = [4, 10, 10, 6, 2]
Output: 2
Explanation:
The array is not organized since *items[2]* mod 2 = *items[3]* mod 2 (both are even). One way to organize the array is:

1. Choose the second index and divide it by 2; the new array is [4, **5**, 10, 6, 2].
2. Choose the fourth index and divide it by 2; the new array is [4, 5, 10, **3**, 2].

The array is now organized since adjacent elements have different parity.

Example 2
Suppose *items* = [6, 5, 9, 7, 3]
Output: 3
Explanation:
The array is not organized. One way to organize the array is:

1. Choose the third index and divide it by 2; the new is [6, 5, **4**, 7, 3].
2. Choose the fifth index and divide it by 2; *items* = [6, 5, 4, 7, **1**].
3. Choose the fifth index and divide it by 2; *items* = [6, 5, 4, 7, **0**].

Constraints

- $1 \le n \le 10^5$
- $1 \le items[i] < 2^{30}$

Question - 37
Match Outcomes

In a game, there are two players, and there is a number sequence of size *n*. Players alternate turns for *n* rounds. Each round, a player removes the first number from the sequence and adds its value to their score. After that, if the removed number is even, the remaining sequence is reversed.

Determine the difference in scores between the two players after the game.

More precisely, suppose *first_score* and *second_score* are the final scores of the first and second player, respectively. The goal is to calculate the value of *first_score - second_score*.

Example
Suppose there are *n* = 5 rounds, and *numSeq* = [3, 6, 2, 3, 5].
Output: 1
Sequence of rounds:

1. The first player picks 3, *first_score* = 3. The remaining sequence: [6, 2, 3, 5].
2. The second player picks 6, *second_score* = 6. Since 6 is even, the remaining sequence is reversed: [5, 3, 2].
3. The first player picks 5, *first_score* = 3 + 5 = 8. The remaining sequence: [3, 2].
4. The second player picks 3, *second_score* = 6 + 3 = 9. The remaining sequence: [2].
5. The first player picks 2, *second_score* = 8 + 2 = 10. The remaining sequence: [].

The difference between players' scores is *first_score - second_score* = 10 - 9 = 1.

Constraints

- $1 \le n \le 10^5$
- $1 \le numSeq[i] \le 10^4$ for each $0 \le i < n$

Question - 38
String Comparison

Given two strings, the objective is to determine if they can be made identical by performing one of the following operations at most once:

1. Select two characters, char1 and char2, and replace all occurrences of char1 in the first string with char2.
2. Select two characters, char1 and char2, and replace all occurrences of char1 in the second string with char2.

For example, the string "xxyz" can be transformed into "yyyz", "vvyz", "xxzz", "xxwz", etc., by performing the above operations. However, it cannot be converted to "yzyz" because all occurrences of a character must be replaced by a single new character.

It is important to note that both operations cannot be performed simultaneously. For instance, given the strings "xz" and "zx", you cannot convert both 'x's in the respective strings to 'z' to make them equal.

Given two arrays of n strings each, compare the strings at the same indices in both arrays and determine if they can be made equal. Return an array of *n* strings, where the ith element is "YES" if the strings at index *i* can be equalized, otherwise return "NO".

Example

firstStrings = ["aaa", "abbc", "zyz"]
secondStrings = ["bbb", "cccc", "zyx"]

Compare the strings at each index:

- *i* = 0: *firstStrings[0]* = "aaa", *secondStrings[0]* = "bbb"
  We can change all the occurrences of 'a' to 'b' in the first string in this case. Therefore, the answer is "YES".
- *i* = 1: *firstStrings[1]* = "abbc", *secondStrings[1]* = "cccc"
  There is no way in which we can make these strings identical. Therefore, the answer is "NO".
- *i* = 2: *firstStrings[2]* = "zyz", *secondStrings[2]* = "zyx"
  We can replace the only occurrence of 'x' in the second string with 'z' to make them identical. Therefore, the answer is "YES".

Hence, the answer corresponding to each pair of strings is *["YES," "NO," "YES"].*

Function Description

Complete the function *canBeEqualized* with the following parameters:

string firstStrings[n]: the first string of each pair
string secondStrings[n]: the second string of each pair

Returns

string[n]: the answers for each pair of strings

Constraints

- $1 \le n \le 10^3$
- $1 \le$ *length of firstStrings[i], length of secondStrings[i]* $\le 10^3$
- All the strings consist of lowercase English letters only.

▼ Input Format For Custom Testing

The first line contains an integer, *n*, denoting the number of elements in *firstStrings*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains a string describing *firstStrings[i]*.
The next line contains an integer, *n*, denoting the number of elements in *secondStrings*.

Each line *i* of the *n* subsequent lines (where *0 ≤ i < n*) contains a string describing *secondStrings[i]*.

▼ second stringsSample Case 0

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
2        →    firstStrings[] size n = 2
ac       →    firstStrings = ["ac", "pqqrr"]
pqqrr
2        →    secondStrings[] size n = 2
ca       →    secondStrings = ["ca", "prrrr"]
prrrr
```

Sample Output

```
NO
YES
```

Explanation

- We can check that there is no way to make them equal for the first pair of strings. Thus the answer is "NO".
- For the second pair of strings, we can convert all the occurrences of 'q' in the first string to 'r', thus the answer is "YES".

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN         FUNCTION
-----         --------
2        →    firstStrings[] size n = 2
a        →    firstStrings = ["a", "aaabbb"]
aaabbb
2        →    secondStrings[] size n = 2
b        →    secondStrings = ["b", "abbbbb"]
abbbbb
```

Sample Output

```
YES
NO
```

Explanation

- For the first pair of strings, we can convert the only occurrence of 'a' to 'b', thus the answer is "YES".
- For the second pair of strings, it is impossible to make the two strings equal, thus the answer is "NO".

Question - 39
K-th Smallest Term

Given an array *arr* of *n* elements, find the kth lexicographically smallest pair in the cross-product array. The cross-product array consists of all possible pairs *(arr[i], arr[j])* for all *i, j* where *0 ≤ i, j < n*.

Note: An element *(a, b)* is lexicographically smaller than element *(x, y)* if:

- *a < x*, or
- *a = x* and *b < y*

Example

There are *n = 3* elements in *arr = [3, 1, 2]*, and *k = 7*.

The cross product array, *cp_arr* = [(3, 3), (3, 1), (3, 2), (1, 3), (1, 1), (1, 2), (2, 3), (2, 1), (2, 2)].

When sorted, *cp_arr* = [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)]. Using 1-based indexing, the kth element, element 7, is (3, 1). Return [3, 1].

Function Description

Complete the function *getkthSmallestTerm* in the editor with the following parameters:

int arr[n]: an array of integers

long_int k: a long integer

Returns

[int, int]: a list of 2 integers denoting the kth lexicographically smallest element of the cross product array.

Constraints

- $1 \le n \le 10^5$
- $1 \le arr[i] \le 10^9$
- $1 \le k \le n^2$

▼ Input Format For Custom Testing

The first line contains an integer, *n*. Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *arr[i]*. The last line contains an integer, *k*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
2        →     arr[] size n = 2
4        →     arr = [4, 1]
1
3        →     k = 3
```

Sample Output

```
4
1
```

Explanation

cp_arr = [(4, 4), (4, 1), (1, 4), (1, 1)]

Sorted, *cp_arr* = [(1, 1), (1, 4), (4, 1), (4, 4)]. Return the 3rd element, (4, 1).

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN          FUNCTION
-----          --------
3        →     arr[] size n = 3
2        →     arr = [2, 2, 1]
2
1
5        →     k = 5
```

Sample Output

```
2
1
```

Explanation

cp_arr = [(2, 2), (2, 2), (2, 1), (2, 2), (2, 2), (2, 1), (1, 2), (1, 2), (1, 1)]

Sorted, *cp_arr* = [(1, 1), (1, 2), (1, 2), (2, 1), (2, 1), (2, 2), (2, 2), (2, 2), (2, 2)]. The 5th element is (2, 1).

Question - 40

Array Reduction 3

Given an array *arr* of *n* integers, a sequence of *n-1* operations must be performed on the array.

In each operation,

- Remove the minimum and maximum elements from the current array and add their sum back to the array.
- The cost of an operation, *cost = ceil((minimum_element + maximum_element) / (maximum_element-minimum_element + 1))* .

Find the total cost to reduce the array to a single element.

Example

Suppose *arr* = [2, 3, 4, 5, 7].

Output: 8

The sequence of operations is:

1. Choose 2 and 7, the *cost = ceil[(2 + 7) / (7 - 2 + 1)] = ceil[9 / 6] = 2*. Remove 2 and 7, append 9, *arr = [3, 4, 5, 9]*, *total_cost* = 2.
2. Choose 3 and 9, the *cost = ceil[(3 + 9) / (9 - 3 + 1)] = ceil[12 / 7] = 2*, *arr* = [4, 5, 12], *total_cost* = 2 + 2 = 4.
3. Choose 4 and 12, the *cost = ceil[(4 + 12) / (12 - 4 + 1)] = ceil[16 / 9] = 2*, *arr* = [5, 16], *total_cost* = 4 + 2 = 6.
4. Choose 5 and 16, the *cost = ceil[(5 + 16) / (16 - 5 + 1)] = ceil[21 / 12] = 2*, *arr* = [21], *total_cost* = 6 + 2 = 8.

Constraints

- $2 \le n \le 2 * 10^5$
- $1 \le arr[i] \le 10^6$

Question - 41

Break The Bricks

There are *n* bricks lined up in a row, numbered from 1 to *n*. Each brick requires a specific amount of force, measured in Newtons, to be smashed, represented by an array *newtons[n]*.

You have two hammers:

- A big hammer that can smash any brick with a single blow
- A small hammer that reduces the required Newtons by 1 with each blow

For example, if a brick requires 3 Newtons, it can be smashed with 1 blow from the big hammer or 3 blows from the small hammer. The big hammer can only be used a certain number of times.

Determine the following three values:

1. The minimum number of blows needed to smash all the bricks
2. The 1-based indices of the bricks smashed by the big hammer, sorted in ascending order
3. The 1-based indices of the bricks smashed by the small hammer, sorted in ascending order

Return these values as a 2-dimensional integer array: [[*total blows*], [*big hammer indices*], [*small hammer indices*]]. If a hammer is not used, its index array should be [-1].

Example

bigHits = 0

newtons = [2]

The big hammer cannot be used. The small hammer takes 2 blows to smash the single brick at index 1. The return array is [[2], [-1], [1]].

bigHits = 4

newtons = [3, 2, 5, 4, 6, 7, 9]

In this case, it is best to use the big hammer on bricks at sorted indices [3, 5, 6, 7], using 4 hits to smash them all. The small hammer is used on sorted indices [1, 2, 4] which have Newtons of 3, 2, and 4. It takes a total of 3 + 2 + 4 = 9 hits with the small hammer. The total blows required = 4 + 9 = 13. The return array is [[13], [3, 5, 6, 7], [1, 2, 4]].

Function Description

Complete the function *breakTheBricks* in the editor with the following parameters:

int bigHits: the maximum blows with the big hammer

int newtons[n]: distinct integers representing Newtons required to smash each brick

Returns

long [[1],[p][q]]: in the form [[total hits], [sorted indices for big hammer hits], [sorted indices for small hammer hits]]

Constraints

- $1 \le n \le 2 \times 10^5$
- $0 \le bigHits \le 2 \times 10^5$
- $1 \le newtons[i] \le 10^9$

▼ Input Format For Custom Testing

The first line contains an integer, *bigHits.*

The next line contains an integer, *n,* the number of bricks and the size of *newtons[].*

Each of the following *n* lines contains an integer, *newtons[i].*

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
9        bigHits = 9
8        newtons[] size n = 8
7        newtons = [7, 9, 3, 2, 5, 8, 4, 6]
9
3
2
5
8
4
6
```

Sample Output

```
8
1 2 3 4 5 6 7 8
-1
```

Explanation

There are enough *bigHits* available to smash all of the bricks with the large hammer. The returned array is [[8], [1, 2, 3, 4, 5, 6, 7, 8], [-1]].

▼ Sample Case 1

Sample Input For Custom Testing

```
0
3
10000000
100000000
1000000000
```

Sample Output

```
1110000000
-1
```

```
1 2 3
```

Explanation

Since *bigHits = 0,* the big hammer cannot be used. The total hits required is the sum of the *newtons* array, one hit with a small hammer for each Newton. The returned array is [[1110000000], [-1], [1, 2, 3]].

Question - 42

Pass the Baton

There are *n* friends standing in a line, each numbered from 1 through *n*. The first friend, friend 1, holds a baton. Each second, the baton is passed to the next friend in line. Once it reaches the last friend, the direction of passing is reversed and continues. Determine who will pass and who will receive the baton at a given time.

Example

Suppose *friends* = 4 and *time* = 5.

Output: [3, 2]

Friends are numbered 1 through 4. Friend 1 holds the baton at time 0. At time 1, it is passed to friend 2. Over 5 seconds, the baton is passed as 1→2→3→4→3→2. The friend passing the baton at time 5 is friend 3. The friend receiving the baton is friend 2.

Constraints

- $2 \leq friends \leq 2 \times 10^5$
- $1 \leq time \leq 10^{12}$

Question - 43

How Many Flips?

You start with a binary string of all zeros. You can perform a flip operation defined as follows:

- Choose any index in the string.
- The digit at that position and all digits to its right toggle (0 → 1, 1 → 0).

Given a target binary string:

- Determine the minimum number of flips required to transform the initial all-zero string into the target string.

Example 1

Suppose *target* = "01011".

Output: 3

Start with a string of 5 zeros, the same length string as the *target*.

0. Initial String -> "00000"
1. Flip the 3rd digit -> "00111"
2. Flip the 2nd digit -> "01000"
3. Flip the 4th digit -> "01011"

Example 2

Suppose *target* = "1010".

Output: 4

Start with a string of 4 zeros, the same length string as the *target*.

0. Initial String -> "0000"
1. Flip the 4th digit -> "0001"
2. Flip the 3rd digit -> "0010"
3. Flip the 2nd digit -> "0101"
4. Flip the 1st digit -> "1010"

Constraints

- 1 ≤ length of *target* ≤ 10^5
- The target string consists of digits 0 and 1.

Question - 44
Missed Chapters

A Chemistry teacher organizes lectures where each chapter is taught in sequence, one per day, repeating from the beginning after reaching the end of the book. Formally, with *numChapters* in the book, on the ith day (zero-indexed), the lecture covers chapter *i % numChapters* where % is the modulo operator.

A student will miss classes from *firstDay* to *lastDay,* inclusive. Calculate the number of different chapters the student will miss lectures on during this absence.

Example

Suppose *numChapters* = 4 *firstDay* = 3, and *lastDay* = 5.

Output: 3

Explanation:

- The series of lectures is on chapters *class* = [0,1, 2, 3, 0, 1, 2, 3,...] starting from day 0.
- For *class[3]* through *class[5],* lectures are given on chapters 3, 0, and 1.
- The student will miss lectures on 3 chapters.

Constraints

- 1 ≤ *numChapters* ≤ 10^9
- 1 ≤ *firstDay* ≤ *lastDay* ≤ 10^9